
ArmPi Ultra

Voice-Controlled Robotic Manipulation

A complete, step-by-step roadmap from zero to a working end-to-end embodied AI system.

Target hardware	Hiwonder ArmPi Ultra (Raspberry Pi 5 + STM32, 6 servos, Aurora depth camera, WonderEch
Goal	Speak a command → robot perceives the scene → robot executes the task
Approach	Modular pipeline (primary deliverable) + optional VLA fine-tuning (stretch goal)
Estimated duration	10–12 weeks at ~10–15 hrs/week
Prerequisite level	None assumed — every concept is explained from first principles

How to use this document: Work through it linearly. Each phase has explicit **exit criteria** — do not move to the next phase until they are met. Code snippets are starting points, not finished products; adapt them to your environment. Anything wrapped in *italics* is a concept you should learn before continuing.

Contents

- 0. Honest Framing — Modular Pipeline vs. End-to-End VLA
- 1. Phase 0: Prerequisite Knowledge (Week 0–1)
- 2. Phase 1: Get the Robot Moving (Week 1–2)
- 3. Phase 2: Vision — Camera, OpenCV, Depth, YOLO (Week 3–4)
- 4. Phase 3: Voice — Speech to Text (Week 5)
- 5. Phase 4: Integration — The Modular VLA Pipeline (Week 6–8)
- 6. Phase 5: Testing, Evaluation, and the Final Report (Week 9–10)
- 7. Phase 6 (Stretch): Real VLA Fine-Tuning
- 8. Engineering Practices You Must Adopt
- 9. Common Failure Modes and How to Debug Them
- 10. Talking to Your Supervisor
- Appendix A: Glossary
- Appendix B: Recommended Resources

0. Honest Framing

Before any technical steps, you must understand what you are actually building. Your original proposal described a **Vision-Language-Action (VLA) model** — a single neural network that takes in an image plus a text command and directly outputs joint commands for the robot. That is a research-grade machine learning problem.

The ArmPi Ultra, as shipped, is built for a different (and simpler) architecture: a **modular pipeline** where speech, language understanding, perception, and motion are handled by separate components glued together. Hiwonder’s built-in tutorials (Chapters 10 and 11 of their wiki) walk you through exactly this approach using ChatGPT, YOLOv8, and classical inverse kinematics.

The two architectures, compared

	Modular pipeline (recommended)	End-to-end VLA
What it is	Separate components for speech, reasoning, perception, and motion.	One neural network takes an image and text directly to joint commands.
Hardware needs	Runs on the Pi 5 you already have.	Needs a desktop GPU for training; inference can sometimes be done on Pi.
Data needs	None for the baseline. Pretrained models do the heavy lifting.	Needs thousands of demonstration trajectories collected on YOUR robot.
Effort	Achievable in ~8–10 weeks as a beginner.	Months of ML work, often unsuccessful on first attempt.
Failure modes	Each stage fails independently — easy to debug.	Whole system fails opaquely — hard to debug.
Matches your proposal	Captures the GOAL (“hear → see → act”).	Captures the literal architecture in the proposal.

Recommendation: Build the modular pipeline as your **primary deliverable**. If time permits, use it to collect demonstrations and fine-tune a small open VLA (such as Hugging Face *Smo/VLA* or *OpenVLA*) as a **stretch goal**. This gives your final report a real comparison — baseline modular vs. learned end-to-end — which is a much stronger research story than running one pretrained model.

1. Phase 0: Prerequisite Knowledge

Estimated time: 1–2 weeks. Do this **before** you start tinkering with the robot.

1.1 Python

You need to be comfortable with: classes and objects, virtual environments (*venv* or *conda*), installing packages with *pip*, reading library source code, and basic exception handling. If you cannot write a Python class that opens a file, processes it, and saves the result, pause here and complete the official Python tutorial first.

Concrete test:

- bullet Create a virtual environment named *armpi*.
- bullet Activate it, install *numpy* and *opencv-python*.
- bullet Write a script that loads a JPEG, converts it to grayscale, and saves the result.
- bullet If any of those steps required Googling more than once, do a Python crash course before continuing.

1.2 Linux command line

The Raspberry Pi 5 inside the ArmPi Ultra runs Linux (Raspberry Pi OS, Debian-based). You will spend most of your time SSH-ing into it. You must know:

Command	What it does	When you use it
<code>ssh user@ip</code>	Open a remote shell on the Pi	Every single session
<code>scp file user@ip:/path</code>	Copy files to or from the Pi	Deploying code
<code>cd / ls / pwd</code>	Navigate the filesystem	Constantly
<code>nano filename</code>	Edit a text file in-terminal	Quick config edits
<code>sudo apt update / install</code>	Install system packages	Setting up dependencies
<code>chmod / chown</code>	Change file permissions	When scripts “won’t run”
<code>systemctl status / restart</code>	Manage background services	Debugging boot-time services
<code>journalctl -u service</code>	Read service logs	Debugging startup failures
<code>top / htop</code>	See running processes	When the Pi is slow
<code>ps aux grep X</code>	Find a specific process	Killing a stuck program
<code>tail -f logfile</code>	Watch a log update live	Debugging running programs

1.3 ROS 2 (Robot Operating System)

The ArmPi Ultra runs **ROS 2**. Despite the name, ROS is not an operating system — it is a middleware framework that lets independent programs (called *nodes*) communicate with each other by publishing and subscribing to *topics* (like named message channels).

Five concepts you must understand:

- bullet **Node** — a running program (e.g. *servo_controller_node*, *camera_node*).
- bullet **Topic** — a named stream of messages (e.g. */camera/image_raw*).
- bullet **Publisher / Subscriber** — one node sends messages on a topic; others receive them.

- bullet **Service** — a request/response call between nodes, like a function call across the network.
- bullet **Launch file** — a Python or XML file that starts multiple nodes together with the right parameters.

Required action:

Complete the official ROS 2 Humble tutorials, sections “Beginner: CLI tools” and “Beginner: Client libraries (Python)”. Budget 6–8 hours. Skip the C++ sections.

```
# Test commands you should be able to run and understand after the tutorials:
ros2 node list
ros2 topic list
ros2 topic echo /some_topic
ros2 topic hz /some_topic
ros2 service list
ros2 launch <package> <launch_file.py>
```

1.4 Robotics fundamentals

You do not need a robotics degree. You need three concepts:

Joint space vs. Cartesian space. Joint space = the angle of each motor. Your arm has 6 motors, so a position in joint space is a list of 6 numbers (e.g. $[0^\circ, 45^\circ, -30^\circ, 0^\circ, 90^\circ, 0^\circ]$). Cartesian space = the (x, y, z) position of the gripper in 3D, plus its orientation. Humans think in Cartesian (“move 5cm to the right”); motors operate in joint space.

Forward kinematics (FK). Given joint angles, compute where the gripper is. This is always solvable and unique — it is just a sequence of geometric transformations.

Inverse kinematics (IK). Given a desired gripper pose, compute the joint angles that achieve it. This is the hard direction — there may be multiple solutions, no solutions, or unreachable poses. Hiwonder provides an IK function for the ArmPi Ultra; you do not need to derive it yourself. But you must understand *why* it can fail.

Why this matters: When the arm refuses to reach a position, 9 times out of 10 it is because the IK solver could not find a valid solution — the point is outside the workspace, or behind the base, or requires a joint to exceed its physical limit. You must learn to recognize this failure mode.

1.5 Phase 0 exit criteria

- bullet You can SSH into the Pi from your laptop.
- bullet You can write and run a Python script in a virtual environment on the Pi.
- bullet You can list ROS 2 topics and echo a message stream.
- bullet You can explain joint space vs. Cartesian space to a friend in one minute.

2. Phase 1: Get the Robot Moving

Estimated time: 1–2 weeks. You follow the Hiwonder wiki chapters 1–3 in order. **Do not skip ahead.** The chapters look basic, but they surface hardware bugs that, if ignored now, will haunt you for weeks later.

2.1 Initial setup (Wiki Ch. 1)

Goal: a working Pi 5 you can SSH into, with the arm responding to the Hiwonder mobile app.

Steps:

- 1 **Flash the SD card** with the Hiwonder-provided image (Section 1.11 of the wiki). Use Raspberry Pi Imager or balenaEtcher. Use a card with at least 32GB.
- 2 **Boot the Pi** with the arm powered. Wait 30–60 seconds. The Pi creates its own WiFi hotspot on first boot.
- 3 **Connect to that hotspot** from your laptop. The wiki lists the SSID and password.
- 4 **SSH in** using the credentials in Wiki Section 1.6. Standard: `ssh ubuntu@192.168.149.1` (verify in the wiki).
- 5 **Configure WiFi** to join your home/lab network so you don't have to disconnect from the internet to use the robot. Edit `/etc/netplan/` or the appropriate config file as the wiki instructs.
- 6 **Find the Pi's new IP** on your home network (`ip a` on the Pi, or scan your router).
- 7 **Verify the app works:** install the WonderPi app on your phone, connect, and drive each joint manually. If this fails, debug now — the hardware is broken or miswired.

Common gotcha: If the app cannot find the robot, the Pi is probably still on its own hotspot rather than your home network. Check by SSH-ing in and running `iwgetid` to see which network it joined.

2.2 Servo control (Wiki Ch. 2)

Goal: write a Python script that moves a single motor to a commanded angle.

Key concepts:

- bullet Each servo has an ID (1–6) and an angle range (typically 0–1000 in raw units, or degrees after conversion).
- bullet Servos are commanded via a ROS 2 topic or service — the wiki shows the exact name.
- bullet Each servo has hard mechanical limits. Commanding outside them can damage the servo. Always test new code with low speeds and small ranges first.

Practical milestone:

Write a script `wave.py` that makes the arm wave: move joint 1 to 30°, then -30°, then back to 0°, then close the gripper. If you can do this, you have proven your code can command the hardware.

2.3 Kinematics (Wiki Ch. 3)

Goal: write a function `move_to(x, y, z)` that places the gripper at a Cartesian point in the robot's base frame.

Wiki sections 3.1–3.4 cover homogeneous transforms, DH modeling, forward kinematics, and inverse kinematics. Read all of them, but you do not need to derive the math — you need to know how to **call** the supplied IK function and **interpret** its failures.

Code skeleton you should end up with:

```
from hiwonder_ik import inverse_kinematics # supplied by Hiwonder
from servo_client import set_joint_angles

def move_to(x, y, z, pitch=-90):
    """Move gripper to (x,y,z) in robot base frame, in millimeters."""
    angles = inverse_kinematics(x, y, z, pitch)
    if angles is None:
        raise ValueError(f'Position ({x},{y},{z}) is unreachable.')
    set_joint_angles(angles, duration_ms=1500)

# Test:
move_to(200, 0, 100) # 20cm forward, centered, 10cm up
```

Document this: Once `move_to()` works, measure the robot's **reachable workspace**: try points in a grid pattern and record which succeed. You will reuse this map in Phase 4 to validate that an LLM-suggested target is reachable before commanding motion.

2.4 Phase 1 exit criteria

- bullet **wave.py** works — you can command individual servos from Python.
- bullet **move_to(x, y, z)** works — you can place the gripper at any reachable point.
- bullet You have a documented workspace map (text file with reachable XYZ ranges).
- bullet You have committed all of this to a Git repository.

3. Phase 2: Vision

Estimated time: 2 weeks. The output of this phase is the second pillar of your system: given a camera frame, return a 3D position for any named object in the scene.

3.1 Camera streaming (Wiki Ch. 4)

The ArmPi Ultra ships with an **Aurora 3D depth camera**. It returns two synchronized streams: an RGB image (what a normal camera sees) and a depth image (distance from the camera to each pixel, in millimeters).

Steps:

- 1 Launch the camera node as the wiki shows (Ch. 4.1).
- 2 Verify both streams: `ros2 topic echo /camera/color/image_raw` and `ros2 topic echo /camera/depth/image_raw`.
- 3 Visualize them: use `rqt_image_view` or write a small Python subscriber that uses `cv_bridge` to convert ROS images to OpenCV.
- 4 Confirm the depth image is sensible: hold your hand 30cm from the camera, sample a few depth pixels, expect ~300mm.

3.2 Classical vision with OpenCV (Wiki Ch. 5)

Before deep learning, you must understand simple color-based detection — it is faster, more reliable for known objects, and the perfect baseline.

Pipeline to build:

- 1 Convert the RGB frame from BGR to HSV color space (`cv2.cvtColor`).
- 2 Threshold to isolate a color (e.g. red, blue, green) with `cv2.inRange`.
- 3 Clean up noise with morphological operations (`cv2.erode`, `cv2.dilate`).
- 4 Find contours (`cv2.findContours`) and pick the largest.
- 5 Compute the centroid of that contour — this is the object's pixel position.

Workshop trick: Build a small live HSV-tuning tool: a window with sliders for H/S/V min and max that updates the mask in real time. You will use it on every new lighting condition. The wiki provides a color threshold tool (Section 1.5) — use it.

3.3 Hand-eye calibration (Wiki Ch. 9 — critical)

This is the most important step in the whole project. The camera reports object positions in *camera coordinates*. The robot accepts targets in *robot base coordinates*. You need to compute the transformation that converts one to the other. This is called **hand-eye calibration**.

Two camera mounts, two methods:

- bullet **Eye-to-hand** (camera fixed, looking at the workspace) — you find a single static transform from camera frame to robot base frame.
- bullet **Eye-in-hand** (camera mounted on the gripper) — the transform changes as the arm moves; you compute camera-to-gripper, then chain through the known gripper-to-base transform.

The ArmPi Ultra has a fixed downward-looking camera on the structure, so you will use **eye-to-hand**. The wiki provides the calibration procedure in Ch. 9 — follow it exactly. The result is a 4×4 transformation matrix you save to a file.

Verification protocol:

- 1 Place a known object at five different table positions.
- 2 For each, run vision to get (u, v) pixel $\rightarrow$ (X, Y, Z) camera coords using depth $\rightarrow$ (x, y, z) robot coords using your saved transform.
- 3 Manually measure the object's actual position relative to the robot base with a ruler.
- 4 Compute the error. Acceptable: < 5 mm in each axis. If worse, recalibrate.

Critical warning: If anyone bumps the camera or the robot moves on the table, calibration is invalid. Make recalibration a one-command script. Suspect bad calibration whenever pick-and-place starts missing.

3.4 YOLOv8 object detection (Wiki Ch. 7)

Color thresholding only works for known, single-colored objects. For general voice commands like “pick up the apple,” you need an object detector. The ArmPi Ultra wiki uses **YOLOv8**.

Two paths:

- bullet **Use the pretrained model** — YOLOv8 ships with weights trained on the COCO dataset (80 common object classes: cup, bottle, apple, banana, book, scissors, etc.). For your demo, this is enough.
- bullet **Train a custom model** — needed only if you require objects YOLO doesn't already recognize (lab equipment, specific toys, etc.). Wiki Sections 7.4–7.6 walk through this with the *Ultralytics* library.

Minimal inference loop:

```
from ultralytics import YOLO
import cv2

model = YOLO('yolov8n.pt') # nano model, fastest on Pi

def detect_objects(rgb_frame):
    results = model(rgb_frame, verbose=False)[0]
    detections = []
    for box in results.boxes:
        cls_id = int(box.cls)
        label = model.names[cls_id]
        conf = float(box.conf)
        x1, y1, x2, y2 = box.xyxy[0].tolist()
        cx, cy = int((x1+x2)/2), int((y1+y2)/2)
        detections.append({
            'label': label, 'confidence': conf,
            'bbox': (x1, y1, x2, y2),
            'center_pixel': (cx, cy),
        })
    return detections
```

Performance note: On a Pi 5, YOLOv8n runs at roughly 5–10 FPS. That is fine for a voice-triggered system (you run it once per command, not continuously). If you need continuous tracking, use color thresholding instead — it runs at 30+ FPS.

3.5 The function that ties Phase 2 together

By the end of Phase 2 you should have one master function:

```
def find_object_3d(rgb_frame, depth_frame, target_label):
    """
    Returns (x, y, z) in robot base coords for the named object,
    or None if not detected.
    """
    detections = detect_objects(rgb_frame)
    for d in detections:
        if d['label'] == target_label:
            u, v = d['center_pixel']
            depth_mm = depth_frame[v, u] # depth at that pixel
            X, Y, Z = pixel_to_camera_3d(u, v, depth_mm)
            x, y, z = camera_to_robot(X, Y, Z)
            return (x, y, z)
    return None
```

3.6 Phase 2 exit criteria

- bullet Hand-eye calibration error is below 5mm.
- bullet **find_object_3d()** returns sensible (x, y, z) for at least three test objects.
- bullet You can combine Phase 1 + Phase 2: place a red cup somewhere, run a script, watch the arm pick it up. **This is your no-voice baseline.**

4. Phase 3: Voice

Estimated time: 1 week. The output of this phase is a function that takes audio from the WonderEcho microphone and returns a text transcript.

4.1 Two paths

Wiki Chapter 10 covers the bundled **WonderEcho Pro** voice box. It supports two fundamentally different modes:

	Path A: Hiwonder defaults	Path B: External STT (recommended)
How it works	Wake word → fixed list of pre-programmed commands	Wake word triggers a call to Whisper or OpenAI API
Vocabulary	Limited to commands you program in firmware	Any spoken English (or Chinese, etc.).
Latency	Very fast (~200ms).	1–3 seconds, depending on STT backend.
Effort	Easy. Just edit Hiwonder's firmware (Wiki 10.3)	Medium. Write your own recorder + STT call.
Best for	Quick demos, fallback mode.	Your real deliverable. Use this.

4.2 Implementing Path B (recommended)

Three options for the STT backend:

- bullet **OpenAI Whisper API** — cloud, ~\$0.006 per minute, very accurate, ~1s latency. Easiest.
- bullet **Local Whisper** (the *whisper.cpp* or *faster-whisper* library) — free, runs on the Pi 5 for small models (*tiny* or *base*), ~3s latency for 5s of audio.
- bullet **Vosk** — fully offline, small models, lower accuracy but very fast.

Skeleton (cloud Whisper):

```
import sounddevice as sd
import scipy.io.wavfile as wav
from openai import OpenAI

client = OpenAI(api_key='...')

def record_and_transcribe(duration_s=5, rate=16000):
    print('Listening...')
    audio = sd.rec(int(duration_s*rate), samplerate=rate, channels=1,
dtype='int16')
    sd.wait()
    wav.write('/tmp/cmd.wav', rate, audio)
    with open('/tmp/cmd.wav', 'rb') as f:
        result = client.audio.transcriptions.create(
            model='whisper-1', file=f, language='en')
    return result.text
```

Trigger logic: You probably want a wake word (“Hey robot...”) before you start recording. The simplest approach: use the WonderEcho’s built-in wake-word detection in firmware to fire a GPIO trigger or send a ROS message; have your Python script subscribe to that and only record/transcribe when triggered. The wiki shows how to wire this in Section 10.5.

4.3 Phase 3 exit criteria

-
- bullet You can say “pick up the red block” into the WonderEcho and see the text transcript on your console.
 - bullet The whole record + transcribe loop completes in under 5 seconds.
 - bullet You’ve tested with at least 3 different speakers and 5 different phrasings.

5. Phase 4: Integration — The Modular Pipeline

Estimated time: 2–3 weeks. This is the core deliverable. You glue Phases 1–3 together with one new piece: an LLM-based **reasoning stage** that converts free-form speech into structured robot actions.

5.1 System architecture

Data flow, end to end:

Stage	Input	Output	Component
1. Speech-to-Text	Audio from WonderEcho	Text string	Whisper (Phase 3)
2. Scene perception	Current camera frames	List of detected objects + 3D positions	YOLOv8 + depth + calibration (Phase 2)
3. Reasoning (LLM)	Text + scene description	Structured action (JSON)	GPT-4o-mini or Claude or local Llama
4. Motion planning	Target 3D position	Sequence of joint waypoints	Hiwonder IK (Phase 1)
5. Execution	Joint waypoints	Physical motion	Servo controller (Phase 1)
6. Verification	Post-action camera frame	Success / failure flag	Phase 2 vision again

5.2 The LLM reasoning stage

This is the new piece. Its job is to convert a free-form sentence + a scene description into a structured action the rest of your code can execute. The trick: force the LLM to output JSON. Modern APIs (OpenAI, Anthropic) support strict JSON mode.

System prompt template:

```
You are the reasoning module of a robotic arm. Given a user's spoken command and the current scene, output a single JSON object describing the action to perform.
```

```
Available actions:
```

- pick_and_place: pick up an object and place it somewhere
- pick: just pick up an object (hold)
- point: point at an object without grasping
- none: command is ambiguous or impossible

```
Available objects in the scene right now:
```

```
{scene_description} # e.g. ['red_cup', 'blue_book', 'apple']
```

```
Output schema:
```

```
{
  "action": "pick_and_place" | "pick" | "point" | "none",
  "target": <object_label> or null,
  "destination": "left" | "right" | "forward" | "back" | null,
  "reason": <one sentence explaining the choice>
}
```

```
If the user refers to an object not in the scene, return action="none".
Do not invent objects. Output only the JSON object, nothing else.
```

Sample call:

```
import json
from openai import OpenAI
client = OpenAI()

def reason(user_text, scene_objects):
    response = client.chat.completions.create(
        model='gpt-4o-mini',
        response_format={'type': 'json_object'},
        messages=[
            {'role': 'system', 'content': SYSTEM_PROMPT.format(
                scene_description=scene_objects)},
            {'role': 'user', 'content': user_text},
        ],
    )
    return json.loads(response.choices[0].message.content)
```

5.3 The main loop

```
def main_loop():
    while True:
        # 1. Wait for wake word + listen
        text = record_and_transcribe()
        print(f'Heard: {text}')

        # 2. Snapshot scene
        rgb, depth = capture_frames()
        detections = detect_objects(rgb)
        scene = [d['label'] for d in detections]

        # 3. Reason
        action = reason(text, scene)
        print(f'Decided: {action}')

        # 4. Execute
        if action['action'] == 'none':
            say('I am not sure how to do that.')
            continue

        target_pos = find_object_3d(rgb, depth, action['target'])
        if target_pos is None:
            say(f'I cannot see the {action["target"]}.'.)
            continue

        execute_pick(target_pos)
        if action['action'] == 'pick_and_place':
            dest_pos = direction_to_position(action['destination'])
            execute_place(dest_pos)

        # 5. Verify
        post_frame, _ = capture_frames()
        if verify_success(post_frame, action):
            say('Done.')
        else:
            say('I think I failed. Sorry.')
```

5.4 The helper functions you need to write

Function	Purpose
<code>capture_frames()</code>	Return synchronized (rgb, depth) numpy arrays from the camera.
<code>execute_pick(xyz)</code>	Open gripper, approach above target, descend, close, lift. Use a pre-grasp pose 10cm above.
<code>execute_place(xyz)</code>	Move above destination, descend, open gripper, retreat.
<code>direction_to_position(name)</code>	Map 'left'/'right' etc. to a fixed (x,y,z) drop zone you defined.
<code>verify_success(frame, action)</code>	Run detection. If the target is no longer at its original location (for a pick) or appears at the
<code>say(text)</code>	Use the WonderEcho's TTS (Wiki 10.5) or a Python TTS library (e.g. pyttsx3).

5.5 Phase 4 exit criteria

- bullet You say “pick up the red cup and put it on the left” — it does.
- bullet You say “grab the blue thing” — it picks up the blue object (LLM disambiguates from scene context).
- bullet You say “pick up the elephant” (which is not in the scene) — it politely refuses.
- bullet End-to-end latency from end of speech to start of motion is under 8 seconds.
- bullet The whole loop is recorded on video for your final report.

6. Phase 5: Testing and the Final Report

Estimated time: 2 weeks. This is where you turn a working demo into a publishable result. Your proposal already specifies the testing strategy — execute it rigorously.

6.1 Experimental matrix

Test each axis of variation independently, then in combination. Record every trial in a spreadsheet — you will need the numbers for the report.

Variation axis	Levels	What you measure
Object position	5 positions in a grid covering the workspace	Pick success rate; XY error at grasp
Object identity	5 “trained” (in YOLO classes) + 3 “novel” objects	Detection rate; pick success
Lighting	Bright / dim / mixed shadows	Detection rate; whether color thresholds still work
Command phrasing	10 paraphrasings per task (e.g. ‘grab’, ‘pick up’, ‘grasp’, ‘fetch’)	LLM stage success rate
Speaker variation	3 different speakers	Transcription accuracy
Distractor objects	1, 3, 5 objects in scene	Disambiguation accuracy

6.2 Latency budget

For each successful run, log how long each stage took. The total is one of your two main report metrics (the other is success rate). A typical breakdown to aim for:

Stage	Target latency	Why it matters
Speech recording	2–5 s (depends on utterance)	User-facing wait time
Speech-to-text	0.5–2 s	Critical for responsiveness
Scene perception (YOLO + depth)	0.2–1 s	Run once per command
LLM reasoning	0.5–2 s	Cloud round-trip
IK + motion planning	0.05 s	Negligible
Physical motion	3–6 s	Servo speed limit
Verification	0.3 s	Optional but valuable
TOTAL	6–16 s	End-to-end UX number

6.3 Failure analysis (the most valuable section of your report)

When the system fails, classify **which stage failed**. Build a confusion table showing failure counts per stage. This is the section that turns your project from “a demo that works” into “research.”

- bullet **Voice failure** — wake word missed, or transcription wrong (e.g. “red” → “dread”).
- bullet **Reasoning failure** — transcript was correct but LLM picked the wrong action or target.
- bullet **Detection failure** — object was in view but YOLO missed it or labeled it wrong.
- bullet **Localization failure** — object detected but its 3D position was wrong (calibration drift, depth noise).
- bullet **Motion failure** — the IK solver returned a pose, but the arm could not reach it physically.
- bullet **Grasp failure** — the arm reached the target but the gripper missed.

6.4 Report structure (matches your proposal's Section 4)

- 1 **Introduction** — problem, gap, what your system does.
- 2 **System Design** — the architecture diagram from §5.1, with photos of the hardware.
- 3 **Implementation** — brief description of each stage, with code references.
- 4 **Experimental Setup** — the matrix from §6.1, the lighting/object setup, the speaker pool.
- 5 **Results** — success rate tables, latency breakdowns, the failure analysis.
- 6 **Discussion** — what worked, what didn't, what surprised you.
- 7 **Limitations and Future Work** — here is where the VLA stretch goal lives.
- 8 **Conclusion.**

7. Phase 6 (Stretch): Real VLA Fine-Tuning

Only attempt this if (a) Phases 1–5 are complete and rock-solid, (b) you have access to a GPU with at least 16GB VRAM, and (c) you have at least 3 weeks left. This is where you honor the literal “VLA model” framing of your proposal.

7.1 What this involves

- 1 **Pick a small open VLA model.** Hugging Face’s *SmolVLA* (designed for small arms, ~450M parameters) is the best fit for your hardware. *OpenVLA* (~7B) is larger but better-known.
- 2 **Collect demonstrations.** Use your modular pipeline from Phase 4 to execute tasks. Record (image, language, joint trajectory) tuples. Target: 50–200 successful demos covering varied objects and positions.
- 3 **Convert to LeRobot dataset format.** Hugging Face’s *LeRobot* library defines a standard schema; *SmolVLA* expects this format.
- 4 **Fine-tune.** Run the LeRobot training script with your dataset. A few hours on a single GPU.
- 5 **Deploy.** Run inference on the Pi 5 (or stream from a desktop). Compare to the modular baseline on the same test matrix from §6.1.

7.2 What the comparison gives your report

- bullet Does the learned VLA **generalize better** to novel object positions?
- bullet Does it handle **novel phrasings** the modular LLM stage missed?
- bullet How does its **latency** compare (typically faster: no LLM round-trip)?
- bullet What is its **failure mode signature**? (Usually all-or-nothing, vs. modular’s stage-specific failures.)

Realistic expectation: A small VLA fine-tuned on ~100 demos will usually *underperform* the modular baseline on the exact tasks you trained, but show interesting failure modes and occasional surprising generalization. That is a perfectly publishable result.

8. Engineering Practices You Must Adopt

These are not optional. Skipping them turns a 10-week project into a 20-week project.

Practice	What and why
Git from day one	Run <code>git init</code> in your project folder before writing any code. Push to GitHub. Commit at least once a day.
Two-machine workflow	Edit code on your laptop (VS Code with Remote-SSH extension is ideal). Push to GitHub. Pull on the Pi.
Virtual environments	One <code>venv</code> per phase. Never <code>sudo pip install</code> globally on the Pi — you will break the system.
Structured logging	Every trial: save the audio, the transcript, the LLM JSON, the detected objects, the planned trajectory.
Test scripts at multiple levels	Unit-level (does <code>move_to(200,0,100)</code> work?), integration-level (does <code>find_object_3d</code> work?), system-level (does the whole thing work?).
Calibration recovery	<code>recalibrate.py</code> as a single command. Print the camera-to-robot transform whenever the system recalibrates.
E-stop	25kg-cm servos can crush fingers and crack tabletops. Have a physical power kill switch within reach.
Configuration in files, not code	Workspace bounds, drop zone coordinates, API keys, camera transform — all in a YAML config file.
Document as you go	Maintain a <code>JOURNAL.md</code> in the repo. Date-stamped notes: what you tried, what worked, what failed.

9. Common Failure Modes and How to Debug Them

Symptom	Most likely cause	How to debug
Arm doesn't move at all	Servo torque disabled, or wrong topic name	Check the topic list; manually publish a test message; check power
Arm twitches violently then stops	Commanded position outside joint limits	Print the joint angles before commanding; clamp them
Arm consistently misses by a fixed offset	End effector calibration drift	Recalibrate; check the camera hasn't been bumped
Arm misses randomly by varying amounts	Depth noise; reflective object	Average depth over a 5x5 pixel patch; matte tape on the object
YOLO detects nothing	Wrong color space, or object not in COCO classes	Save the detected frame and run YOLO on it manually; check it's BGRA
Voice transcript is garbled	Background noise; microphone gain too low	Record a 10s sample and listen back; adjust WonderEcho gain
LLM returns malformed JSON	Prompt is too long, or temperature too high	Use strict JSON mode in the API; set temperature=0
LLM picks the wrong object	Scene description was incomplete	Log the exact scene description sent to the LLM; include color + pose
IK function returns None	Target is outside reachable workspace	Check the workspace map from Phase 1; clip targets to the reachable area
System slows down over time	Memory leak in a long-running ROS node	Stop the node while running; restart nodes if RSS keeps growing
Pi reboots randomly	Power supply too weak for Pi 5 + sensors supplied	Use the supplied power brick; never USB-power the Pi during arm

10. Talking to Your Supervisor

Your original proposal was written for the SO-ARM101 with a pure VLA model approach. You've switched to the ArmPi Ultra, which is built around a modular architecture. Do not silently switch architectures — raise it early.

10.1 Suggested email template

Subject: VLA Project — Scope Adjustment for ArmPi Ultra: Dear Prof. Nguyen,

I have now received and assembled the ArmPi Ultra, which differs in ecosystem from the SO-ARM101 I described in the proposal. After studying the hardware and Hiwonder's documentation, I want to propose a slight refinement to the project scope.

The ArmPi Ultra is best suited for a **modular voice→LLM→vision→IK pipeline** as the primary deliverable — Hiwonder's toolchain supports this directly, and it lets me focus on integration and evaluation rather than infrastructure. I would then position an **end-to-end VLA fine-tuning experiment** (using SmolVLA on demonstrations collected from the modular system) as a stretch goal in the final 2–3 weeks. This gives the final report a direct comparison between the two architectures — modular vs. learned end-to-end — which I think is a stronger research contribution than either alone.

The proposal's testing methodology (Section 3) remains unchanged. Could we discuss whether this re-scoping is acceptable? Happy to draft a one-page revised plan if useful.

Best regards.

Supervisors prefer this kind of proactive scoping conversation early, when there is still time to adjust, rather than discovering the mismatch in your final report.

Appendix A: Glossary

Term	Definition
Calibration (hand-eye)	The procedure of computing the geometric transformation from camera coordinates to robot base coordinates.
COCO	A standard dataset of 80 common object classes; YOLO models are commonly pretrained on it.
DH parameters	Denavit–Hartenberg parameters — a standard way to describe a robot's link geometry.
Embodied AI	AI that interacts with the physical world through a body (a robot).
Forward kinematics	Computing gripper position from joint angles.
Hand-eye calibration	See “Calibration”.
Inverse kinematics (IK)	Computing joint angles needed to reach a desired gripper position.
Joint space	Coordinate system defined by joint angles.
LeRobot	Hugging Face library for robotics datasets and policies.
LLM	Large Language Model. E.g. GPT-4o, Claude, Llama.
Modular pipeline	An architecture where each capability (speech, perception, reasoning, motion) is a separate component.
Node (ROS)	A running process in a ROS 2 system.
OpenVLA	An open-source 7B-parameter VLA model.
Point cloud	A set of (x, y, z) points representing 3D geometry, often derived from a depth image.
Pose	Position (3D point) + orientation (3D rotation) — the full 6-DOF description of a rigid body.
ROS 2	Robot Operating System version 2 — a middleware framework for robotics.
RGB-D	A camera providing both color (RGB) and depth (D) streams.
SmolVLA	A small open VLA model (~450M params) designed for small robot arms.
STT	Speech-to-Text.
Topic (ROS)	A named message stream in ROS 2.
Trajectory	A sequence of joint configurations over time.
TTS	Text-to-Speech.
VLA	Vision-Language-Action model. A neural network mapping (image, text) directly to robot actions.
Whisper	OpenAI's open-source speech recognition model.
Workspace	The 3D region of space the robot's gripper can reach.
YOLO	“You Only Look Once” — a family of fast object detectors.

Appendix B: Recommended Resources

Hardware-specific (use first)

- bullet **Hiwonder ArmPi Ultra wiki:** <https://wiki.hiwonder.com/projects/ArmPi-Ultra/en/latest/>
- bullet **Hiwonder downloads (firmware, code samples):** the Google Drive folder linked from the wiki index.

ROS 2

- bullet **ROS 2 Humble Tutorials (official):** <https://docs.ros.org/en/humble/Tutorials.html> — do “Beginner: CLI tools” and “Beginner: Client libraries (Python)”.
- bullet **Articulated Robotics YouTube channel:** beginner-friendly ROS 2 explainers.

Computer vision

- bullet **PyImageSearch OpenCV tutorials** — strongest free resource for OpenCV by example.
- bullet **Ultralytics YOLOv8 docs:** <https://docs.ultralytics.com>

Robotics math

- bullet **Modern Robotics (Lynch & Park)** — free textbook and Coursera course. Chapters 1–4 cover what you need.
- bullet **Peter Corke’s Robotics Toolbox tutorials** — hands-on, Python-based.

Voice and LLMs

- bullet **OpenAI API docs:** <https://platform.openai.com/docs> — Whisper, Chat Completions, structured outputs.
- bullet **Anthropic API docs:** <https://docs.claude.com> — if you prefer Claude for the reasoning stage.
- bullet **faster-whisper** (local STT): <https://github.com/SYSTRAN/faster-whisper>

VLA / LeRobot (Phase 6)

- bullet **Hugging Face LeRobot:** <https://github.com/huggingface/lerobot>
- bullet **SmolVLA:** see the LeRobot repo and accompanying Hugging Face blog posts.
- bullet **OpenVLA paper and code:** <https://openvla.github.io/>

Final word

You said you know nothing of the required knowledge. That is fine — nobody starts knowing this. The single most important habit is to **stop and verify each phase’s exit criteria before moving on**. Skipping ahead seems faster; it never is. Build the boring foundation, and the impressive demo at the end will assemble itself.

End of roadmap. This document is a living plan — mark it up, cross things out, and adjust as you learn what your hardware actually does.